

Lecture 21: See the Network

CLI Tools, Wireshark, and pyshark

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Session 1 — CLI Tools

- ping
- traceroute
- ip addr / route / neigh
- ss
- dig
- tcpdump

Session 2 — Analysis

- Wireshark GUI walkthrough
- pyshark: packets as a dataset
- Extract DNS queries
- Plot TCP RTT

Theme

You spent a semester learning what's inside packets. Today you watch them fly.

Why Linux Tools?

- Network engineers diagnose and monitor networks using Linux and BSD tools
- **Juniper JunOS** (backbone routers) runs on FreeBSD
- **Arista EOS** (data center switches) runs on Linux
- **pfSense / OPNsense** (firewalls) run on FreeBSD
- tcpdump, ping, traceroute, ss are standard on all of them

This is not a simulation of professional practice — it is professional practice.

Concepts: ICMP Echo Request/Reply (L17), RTT estimation (L19), TTL (L17)

```
ping -c 5 8.8.8.8  
ping -i 0.2 -c 20 8.8.8.8 # fast ping, watch RTT variance
```

- RTT variation → RTTVAR that TCP tracks
- TTL in reply → count back from 64/128 to estimate hop count

Concepts: TTL decrement (L17), ICMP Time Exceeded (L17), forwarding (L13)

```
traceroute -m 20 8.8.8.8  
traceroute -T -p 80 8.8.8.8 # TCP mode, bypasses ICMP filters
```

- Probes with $TTL = 1, 2, 3, \dots$ — each router reveals itself
- Stars (* * *) = ICMP filtered, not broken
- RTT jumps often mark AS boundary crossings

Concepts: IP addressing (L14), routing table (L13), ARP (L08)

```
ip addr show # interfaces and addresses
ip route show # forwarding table
ip neigh show # ARP cache (IP -> MAC)
```

- ip route: this is the forwarding table the router walks
- ip neigh: ARP cache entries with state (REACHABLE, STALE)

Concepts: TCP sockets (L18), cwnd / ssthresh (L20), RTT (L19)

```
ss -t # established TCP connections
ss -tipm # full TCP internals
# Watch cwnd grow during iperf:
iperf3 -s &
iperf3 -c 127.0.0.1 -t 30 &
watch -n 0.5 'ss -tipm | grep iperf'
```

ss -tipm key fields:

Field	Meaning
cwnd:N	Congestion window (MSS units)
ssthresh:N	Slow-start threshold
rtt:X/Y	Smoothed RTT / RTTVAR (ms)
retrans:N/M	Current / total retransmissions

Concepts: DNS name resolution (preview of L23), UDP (L18)

```
dig google.com # full output
dig +short google.com # just the IP
dig @8.8.8.8 google.com # specify resolver
dig +trace google.com # full resolution chain
dig google.com MX # mail records
```

- TTL in answer section = DNS cache lifetime
- +trace: root → TLD → authoritative (L23 preview)
- Runs over UDP — watch it with tcpdump port 53

Concepts: every protocol we've studied

```
# Watch DNS queries live
sudo tcpdump -i any -n port 53
# Capture to file:
sudo tcpdump -i any -n -w ~/capture.pcap &
ping -c 5 8.8.8.8
dig google.com
curl -s https://example.com > /dev/null
sudo pkill tcpdump
```

Filter syntax:

- host 8.8.8.8 port 443 tcp udp icmp
- tcp and port 80 (compound filters)

Transfer the Capture

Move the pcap from the VM to your Mac for Wireshark:

On your Mac:

```
multipass transfer ec441:/home/ubuntu/capture.pcap ~/capture.pcap
```

Open in Wireshark. We will look at:

- Protocol hierarchy (*Statistics* → *Protocol Hierarchy*)
- A DNS query: Ethernet → IP → UDP → DNS
- A TCP stream: *Analyze* → *Follow* → *TCP Stream*
- SYN / SYN-ACK / ACK sequence numbers

Wireshark

- One conversation at a time
- Click to explore
- Great for diagnosis

pyshark

- Treat a capture as a dataset
- Ask questions across thousands of packets
- Automate analysis, generate plots

Field names in pyshark match Wireshark exactly. If you can see it in the GUI, you can access it in Python.

```
import pyshark
cap = pyshark.FileCapture('/home/ubuntu/capture.pcap')
for pkt in cap:
    print(pkt.highest_layer, pkt.length)
```

```
cap = pyshark.FileCapture('/home/ubuntu/capture.pcap')
pkt = next(iter(cap))
print(pkt.layers) # all layers present
print(pkt.ip.src) # source IP
print(pkt.ip.ttl) # TTL from L17
print(pkt.transport_layer) # TCP or UDP
```

Field names match Wireshark exactly — if you can see it in the GUI, you can read it here.

What domains did this machine look up?

```
cap = pyshark.FileCapture('capture.pcap', display_filter='dns')
queries = []
for pkt in cap:
    try:
        if hasattr(pkt.dns, 'qry_name'):
            queries.append(pkt.dns.qry_name)
    except AttributeError:
        pass
for q in sorted(set(queries)):
    print(q)
```

How did RTT evolve during a connection?

```
import pyshark, matplotlib.pyplot as plt
cap = pyshark.FileCapture('capture.pcap',
    display_filter='tcp.analysis.ack_rtt')
times, rtt = [], []
for pkt in cap:
    try:
        times.append(float(pkt.sniff_timestamp))
        rtt.append(float(pkt.tcp.analysis_ack_rtt) * 1000)
    except AttributeError:
        pass
plt.plot(times, rtt, '.', markersize=3)
plt.xlabel('Time(s)')
plt.ylabel('RTT(ms)')
plt.savefig('rtt_plot.png')
```

This is the data TCP's EWMA is smoothing — see the variance your RTO must account for.

What's Next

Lecture	Date	Topic
L22	Tue Apr 21	Touch the network — Scapy, Mininet, sockets
L23	Thu Apr 23	Application layer — HTTP, DNS, QUIC
L24	Tue Apr 28	Cryptography and security
L25	Thu Apr 30	Project demo day

L22 Preview

Today you read packets. Next week you write them — craft custom packets with Scapy, build topologies with Mininet, write TCP/UDP apps with sockets.